

Index:

Standard Documentation:

1. Product vision
2. Project goals
3. Coding standards
4. Documentation standards
5. Development environment (tech stack)
6. Deployment environment
7. Version management
8. Change management
9. Definition of Done (Revised)

Test Documentation:

1. Tests List
2. Test Automation Method

Design Document:

1. Architectural Design
2. Database Design
3. UI/UX Design

Product Vision:

“To better aid human connection between people in an organized and visually memorable way.”

Project Goals:

- Easy and fun to use front-end
- Full CRUD functionality for person rolodex entries
- Full CRUD functionality for Room folders to organize rolodex entries
- Multiple types of notes to organize detailed information about entries

Coding Standards:

Followed standard conventions for naming with camelCase and PascalCase. YAML files have included hyphens as spaces for legibility, and regions have been nested with “.” to differentiate them, also for readability. Past these details, we just used the best-practice standards for the files and languages employed in the program.

Documentation Standards:

Our documentation existed primarily within comments within the codebase itself. Though there are additional supplemental documents on the user-view of utilizing the application provided on GitHub.

Development Environment:

Our development environment is Visual Studio Community 2026 running .NET 10 in combination with Maui plugins to allow for Android development. We also have heavily employed the use of Android device emulators for testing.

Deployment Environment:

Our primary deployment environment is Android mobile devices, as it is an Android application.

Version Management:

Versioning is maintained in a three-segment code attached with each release 0.0.0, a major release increments the first segment and resets the following two (0.5.7 -> 1.0.0), a minor feature release increments the second segment and resets the following one (1.4.3 -> 1.5.0), and small patches, updates, and bug fixes increment the final segment only (1.5.1 -> 1.5.2).

Change Management

Our change management and bug tracking are maintained by the GitHub software we are using for this project. Using pull and push requests helps minimize merge conflicts, and the Issues tab in GitHub is a quick and efficient way to track bugs that occur during testing.

Definition of Done

All work has been peer reviewed, passed all automated and manual tests without producing errors, and can accomplish the project goals set out in the sprint plans.

Testing Documentation:

Unit Tests Performed on Each Release:

- a. Create Person (Success)
- b. Read Person (Success)
- c. Read Person (Fail)
- d. Read Person (Fail)
- e. Update Person (Success)
- f. Read All Person (Success)
- g. Read All Person (Fail)
- h. Delete Person (Success)
- i. Create Room (Success)
- j. Read Room (Success)

Test Automation:

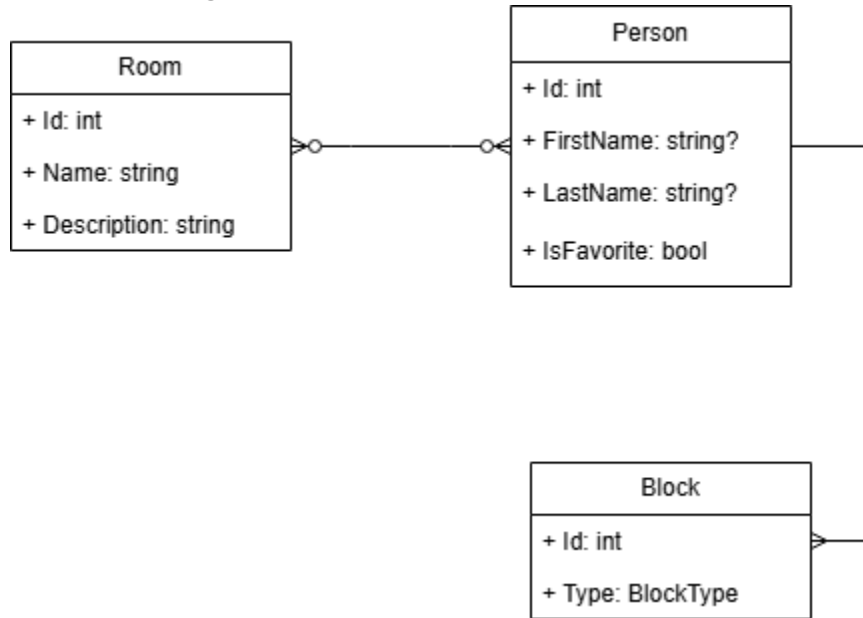
Automation is performed on each release utilizing YAML files to run the testing procedures.

Design Documents:

Architectural Design:

The front end of the application ties directly to a SQL database stored locally within the app. There are no external API or Database connections.

Database Design:



Within the database, a user can create no rooms or many, and within each of these rooms select 0 to many individual rolodex people. These people can belong to multiple rooms as well. Each person, of which 0 to many can exist, will have 0 to many blocks of additional information. These blocks Must be tied to one specific person.

UI/UX Design:

The UI/UX design is primarily based on the Pokedex designs of various older Pokémon games. This is brought out using the sharp geometric shapes in the home screen and retro font choices as well as strong color design with the red in light mode and dark blues in dark mode.